Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

असतो मा सद्गमय

UNIT-3

SEARCHING TECHNIQUES & SORTING TECHNIQUES

Sequential and binary Search, Sorting- Basic concept, Sorting by bubble Sort Insertion Sort, selection Sort, Quick Sort heap Sort, Merge Sort, Radix Sort, and Counting Sort Algorithm.

Searching - Linear search $\Theta(n)$
Binary search $O(\log n)$

Linear search -

Linear-search(Data, N, item, LOC)
Data is linear Array with N elements and item is a given item need to search LOC is return location, if item found else LOC = 0, if the search unsuccessful.

1. ~~Insert item~~ Initialize Counter
   set LOC = 0
2. [search for item]
   Repeat while Data[LOC] ≠ item
   set LOC = LOC + 1
3. [Unsuccessful] If LOC > N then set LOC = 0
4. Exit

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Complexity

In the worst case, one must search through the entire array DATA.

$$f(n) = n+1$$

In particular, suppose q is very small, and item appear with equal probability $q = 0$ and each $p_i = 1/n$

$$f(n) = 1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + \cdots n \cdot \frac{1}{n} + (n+1) \cdot 0 =$$

$$(1 + 2 + 3 + \cdots n) \frac{1}{n}$$

$$n\frac{(n+1)}{2} \cdot \frac{1}{n} = \frac{n+1}{2} = n$$

In the special case, the average number of comparisons required is approximately equal to half the number of element in the array.

Binary search

Binary-search (Data, LB, UB, Item, loc)

1. [Initialize segment variable]
   set BEG = LB, END = UB and MID = INT (BEG + END)/2

2. Repeat step 3 and 4 while BEG ≤ END and Data[MID] ≠ item

3. If Item < Data[MID] then:
   set END = MID - 1
   Else
   set BEG = MID + 1

4. set MID = INT (BEG + END)/2
   End of step 2 loop

5. If Data[MID] = ITEM then
   set loc = MID
   Else
   set loc = NULL

6. Exit

$n/2$
$n/4$
$n/8$
After K - $\frac{n}{2^K} \approx 1$
$2^K = n \rightarrow \log_2 K = \log n$

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Complexity -

Worst case time complexity is $O(\log n)$

Ex Let 1,000,000 element

$2^{10} = 1024 > 1000$ and

$2^{20} > 1000^2 = 1000000$

that means BS Algorithm, require only 20 Comparisons to find location.

Limitation

(1) the list must be sorted
(2) one must have direct access to the middle element in any sublist

Advantage

(1) very efficient
(2) Worst case time complexity $\log n$

SORTING - Bubble Sort

Pass1

	①	②	③	④	⑤	⑥	⑦
8	4	4	4	4	4	4	4
4	8	8	8	8	8	8	8
19	2	19	2	2	2	2	2
2	7	2	19	7	7	7	7
7	13	7	7	19	13	13	13
13	5	13	13	13	19	5	5
5	16	5	5	5	5	19	16
16		16	16	16	16	16	19

Pass2

	①	②	③	④	⑤	⑥
4	4	4	4	4	4	4
8	2	2	2	2	2	2
2	8	8	7	7	7	7
7	7	7	8	8	8	8
13	13	13	13	13	5	5
5	5	5	5	5	13	13
16	16	16	16	16	16	16
19	19	19	19	19	19	19

Pass3

	①	②	③	④	⑤
2	2	2	2	2	2
4	4	4	4	4	4
7	7	7	7	7	5
8	8	8	5	5	7
5	5	5	8	8	8
13	13	13	13	13	13

Pass4

2	2	2	2	2	2
4	4	4	4	4	4
7	7	7	5	5	5
5	5	5	7	7	7
8	8	8	8	8	8

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Bubble sort

Bubble_Sort (Arr, N) .

1. Repeat step 2 and 3 for K = 1 to N-1 //n
2. set PTR = 1
3. Repeat while PTR ≤ N-K //n
   (a) If Arr [PTR] > Arr [PTR+1] then:
   Swap Arr[PTR] & Arr [PTR+1]
   (b) PTR = PTR +1
   End
   End
4. Exit

Complexity

$O(n^2)$ – Average Case

$O(n^2)$ – Worst Case

12 29 25 8 32 17 40

Selection_Sort(Arr, N)

1. Repeat for i = 1 to N-1
2. min j = i // for position
3. min x ← A[i]
4. for j = i+1 to n do
   if A[j] < min x then
   min j = j
   min x = A[j]
5. A[min j] = A[i]
6. A[i] = min x

Complexity

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

असतो मा सद्गमय

Insertion Sort

39 9 45 63 18 81 108 54 72 36

9 39 45 63 18 81 108 54 72 36

9 39 45 63 18 81 108 54 72 36

Pass4 = 9 39 45 63 18 81 108 54 72 36

9 18 39 45 63 81 108 54 72 36

Pass6

Pass7

Pass8 9 18 39 45 54 63 81 108 72 36

Insertion Sort (Arr, N)

1. Repeat step 2 to 5 for K = 1 to N
2. set temp = Arr [K]
3. set J = K-1
4. Repeat while temp <= Arr [J]
   set Arr [J+1] = Arr [J]
   set J = J-1
5. set Arr [J+1] = temp
6. Exit

Quick Sort

- ★ widely used algorithm
- ★ makes O(n log n) comparisons in Average
  $O(n^2)$ in worst case.
- ★ Also known as partition exchange sort
- ★ Work on divide and conquer strategy like merge sort.
- ★ 1) Selet Pivot element
  2) do partition as per pivot
  3) Sort two Subarray
- ★ faster than bubble, selection, insertion,

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

```
Quick - Sort ( ARR, BEG, END )
step1.   If (BEG < END) then
             call partition ( Arr, Beg, End, Loc)
             call Quick Sort ( ARR, BEG, LOC-1)
             Call Quick Sort ( Arr, LOC+1, END)
         End of If
step2-   END

PARTITION ( ARR, BEG, E,    L )
1.  set left = Beg, Right = End , Loc = Beg
    flag = 0
2.  Repeat step 3 to 6 while flag = 0
3.  Repeat while Arr[Loc] <= Arr[Right]
    And Loc != Right
        set Right = Right - 1
4.  If Loc == Right the
        flag = 1
```

elsey Arr[Lor] > Arr[Right] the
swap Arr[Lor] with Arr[R
set Lor = Right

steps If flag = 0 then
Repeal while Arr[Lor] >= Arr[Left]
And Lor != Left
set Left = Left + 1
6 If Lor = = Left then
Flag = 1
Elsey Arr[Lor] < Arr[Left] then
swap Arr[Lor] with Arr[Left]
set Lor = Left
End of If
7. End of loop
7. End

Ex (27) 10 36 18 25 45
25 10 36 18 (27) 45
25 10 (27) 18 36 45
25 10 18 (27) 36 45

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Merge Sort

* Merge sort uses the divide, conquer and combine algorithm
* Divide means - n elements divided into 2 sub-array of $n/2$ until sub array have 1 or zero element
* Conquer - means sorting the 2 subarray recursively using merge-sort
* Combine - means merging the two sorted sub-array of size $n/2$

Ex

39 9 81 45 90 27 72 18

39 9 81 45 90 27 72 18

39 9 81 45 90 27 72 18

↓ ↓ ↓ ↓ ↓ ↓

39 9 81 45 90 27 72 18

9 39 45 81 27 90 18 72

9 39 45 81 27 90 18 7

9 39 45 81 18 27 72 90

9 18 27 39 45 72 81 90

Combine the elements to form a sorted Array

Merge (ARR, BEG, MID, END)

1. Initialize set I = Beg J = Mid+1, Index=0
2. Repeat while (I ≤ Mid) And (J <= END) K=0
3. If ARR[I] < ARR[J] then
   set temp[Index] = Arr[I]
   set I = I+1
   Else
   set Temp[Index] = Arr[J]
   set J = J+1
   K++
4. [Copy the remaining elements of Eight sub-Array, if any]
   If I > MID then
   Repeat while J<=END

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5150400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Repeat while J <= END

. set temp[Index] = Arr[J]

set Index = Index + 1

set J = J + 1

Else

Repeat while I <= MID

set temp[Index] = Arr[I]

set Index = Index + 1

set I = I + 1

step 5. [Copy the contents of temp back to Arr] set K = 0

step 6. Repeat while K < Index

set Arr[K] = temp[K]

set K = K + 1

step 7. End

```
while (i<=mid)
    C[k++] = A[i++]
while (j<=high)
    C[k++] = A[j++]
for (i= low; j=0; i<= high, i++, j++)
  {
    A[i] = C[j]
  }
}

ms(          )
  int mid
{ if (low < high)
  {
    mid = (low+high)/2
    MS (A, low, mid)
    MS (A, mid+1, high)
    m (A, L, h, m)
  }
}
```

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Radix Sort (ARR, N)

1. Find the largest number in Arr as large
2. Initialize set Nop = Number of digit in large
3. set pass = 0
4. Repeat step 5 while pass <= Nop - 1
5. set I = 0 And Initialize bucket
6. Repeat step 7 to 9 while I < N
7. set Digit = digit at Pass th place in A[I]
8. Add A[I] to bucket no. Digits
9. Increment bucket count for bucket numbered Digit
10. Collect the no. in the bucket
11. END

345, 654, 924, 123, 567, 472
555 808 911

0	1	2	3	4	5	6	7	8	9
					345				
				654					
				924					
			123						
							567		
		472							
					555				
911								808	

911, 472, 123, 654, 924, 345, 555, 567, 808
do the simulor. for digit 2nd and 1st.

Complexity - larget digit have - k digit number.
So the outer loop execute k time
the radix sort execute for n time
So the complexity $\Theta(kn)$

- ☆ fastest sorting algorithm
- ☆ It takes more space the other
- ☆ Algorithm is dependent on digit or letter

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Counting Sort -

Array - [1, 2, 4, 3, 1, 5, 8]

Count

0	1	2	3	4	5	6	7	8	9
0	2	1	1	1	1	0	0	1	0

occurance

Output

0	1	2	3	4	5	6	7	8	9
0	2	3	4	5	6	6	6	7	7

position

0	1	2	3	4	5	6	7
1	1	2	3	4	5	8	

Input size is 'n'
Given Range is 'k'

Complexity is $n+k$

Ex-2 1 3 2 3 4 1 6 4 3

0	1	2	3	4	5	6
0	2	1	3	2	0	1

0	1	2	3	4	5	6
0	2	3	6	8	8	9

0	1	2	3	4	5	6	7	8	9
1	1	2	3	3	3	4	4	6	

```
Count Sort (Arr, n)
int k = arr[0]
for (int i = 0; i<n ; i++)
    k = max (k, arr[i])          // n

int count[10] = {0};
for ( int i = 0; i<n ; i++)
    count [ arr[i]] ++;          // n

for ( int i = 1 ; i<= k; i++)
    count[i] + count[i-1];       // k

int output[n];
for ( i = n-1 ; i >= 0; i--)
  output[ --count [ arr[i]]] = arr[i] // n

for ( i = 0 ; i < n; i++)
    ~~cout~~ << arr[i] = output[i];   // n
```

// Youtube / ApnaCollege / Count Sort

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Time Complexity

Total time O(n+k)

* efficient Algorithm
* If the range of input array (k) is very large, the algorithm require a lot of space and become inefficient

Heap Sort

- max heap – Ascending order
- min heap – Decending order

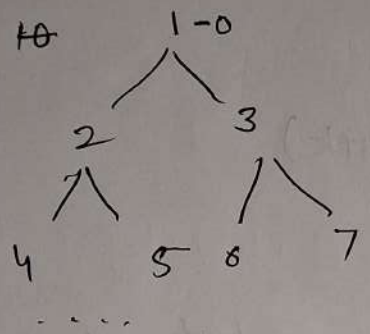
Sorting step -

- Make the max heap for Given Array
- Remove the root element by applying heapify array
- Reduce the size of heap by 1
- Again Heapify the root elem
- Repeat the process until all Sorted

Input Array

Ex 10, 3, 76, 34, 23, 32

① Tree struct



1	2	3	4	5	6	7
0	1	2	3	4	5	6

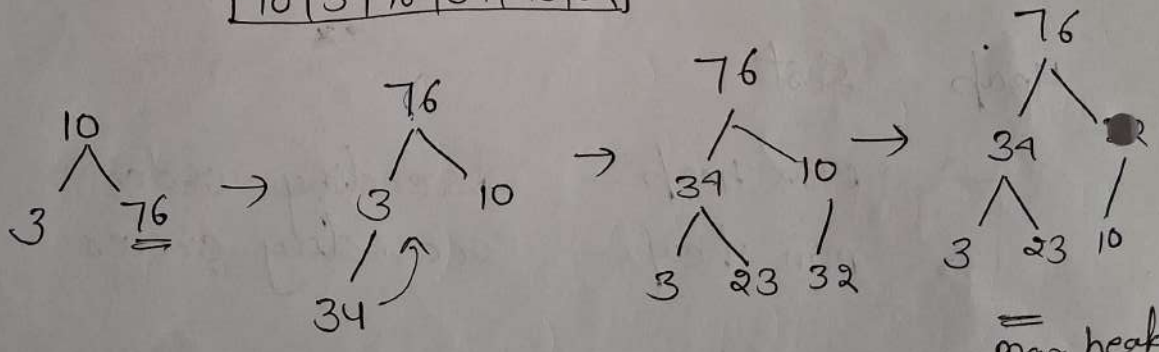
Root – i [=0]

Left → 2i+1

Right → 2i+2

② Build max heap

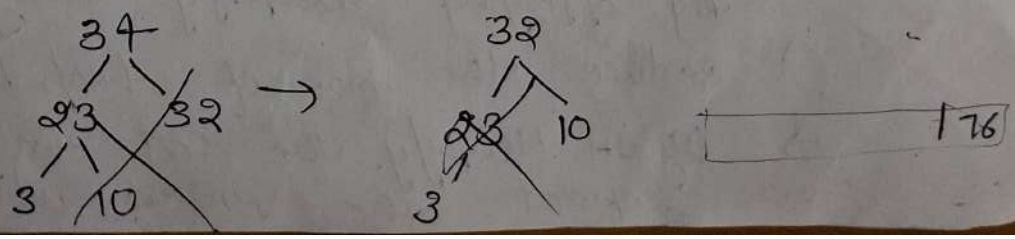
10	3	76	34	23	32



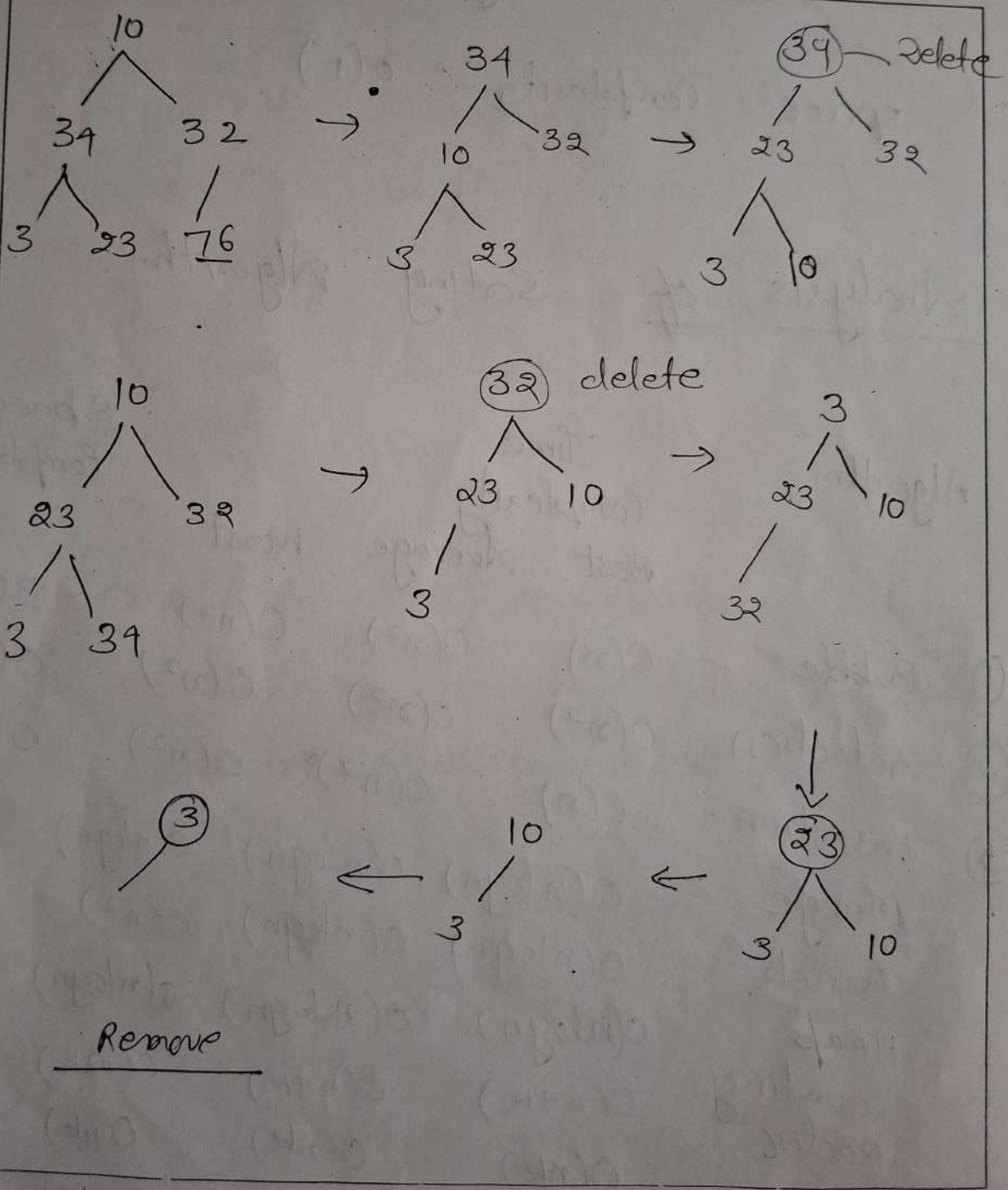
= max heap

76	34	32	3	23	10

③ Apply heapify [Remove Root node and add in the end]



Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in



Complexity of heap sort -

- $O(n \log n)$

space Complexity - $O(1)$

Analysis of Sorting Algorithm

Algorithm	Time Complexity Best	Average	Worst	Space Complexity
(1) Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
(2) selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
(3) Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$
Radix	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

...mand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

असतो मा सद्गमय

One word

Bubble Sort - Swapping
Selection Sort - Swapping
Insertion Sort - Shifting
merge Sort - Allocation of extra memory and data copy.
Quick Sort - Swapping
Heap Sort - Swapping

Internal (In-place) or External Sorting

↳ No extra memory space required
Bubble, selection, insertion, quick, heap

↳ External - merge sort, Counting sort
Radix sort

stable - unstable -

stable if relative order of element is same as in given I/P

stable - Bubble,
Insertion
merge
Counting
Radix

unstable - selection,
quick
heap sort

online vs offline

An algorithm that sorts data as it becomes available is called an online. (Insertion)

offline - algorithm require all input data to be present in memory before sorting can start.
selection.

SKIT
असतो मा सद्गमय
Kesharvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

pairwise swapping

(1) Start with first node of the list
(2) Traverse the list two node at a time
3 for every pair of nodes, swap their link
(4) Move to the next pair and continue until the end of list is reached

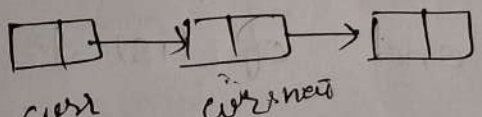
```
Node * pairwiseSwap ( Node * head)
If ( head == NULL or head->next == NULL)
   return head
- newhead = head -> next
- Prev = NULL
- Curr = head
while ( Curr != NULL && Curr -> next != NULL)
{   top next = Curr->next
    top nextNext = top next->next
```

```
temp->next = curr
curr-> next = temp->next

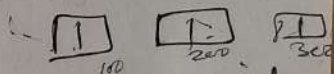
If (PREV != NULL)
{ PREV->Next = next
}

PREV = curr
curr = temp->next

}
return new head
}
```




```
temp = curr->next
curr->next = temp->next
temp->next = curr
~~Head = curr~~
If (PREV != NULL)
    prev->next = temp
else
    Head = curr temp
```




```
next = curr
nextNext = next
SO
next->next = cur
cur->next = nextnext
```

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

```
Node Poly
{
    int coeff.
    int exponent
    Node *link
}
Poly1 →
Poly2 →
Addpoly ( ——  —— )
{
Node* result = Null
while ( poly1 != NULL && poly2 != NULL)
    if ( poly1 → exponent = poly2 → exponent)
        temp = New node
temp → coeff = poly1 → coeff + poly2 → coeff
temp → exponent = poly1 → exponent
    temp → link = Head;
    poly1 = poly1 → next
    poly2 = poly2 → next
```

```
elseif (poly1 -> exponent > poly2 -> expon
{
    poly1 = poly1 -> next
}
else
{
    poly2 = poly2 -> next
}
```

① Palindrome

1. find the middle of the list
2. Reverse the second half
3. Compare the two halves
4. Restore the list
5. Return the Result.

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Hashing

* Linear Search — O(n)
* Binary Search — O(logn)

Is there any way to search an Item in O(1) time.

Two way

(1) If we have 100 employe, and all have emp-Id, from 1 to 100, then keep this emp-Id as Array Index, so we can access data in O(1). time, But what if, if No of employees are 1 Lakh Or emp-Id is 4 or 5 digit number like 00000 to 99999 then Arr[99999] is required. Also if there are only 100 employee But have employee Id (5digit). Observe that, here impractical it is to waste so much storage space.

② way is, If there is 100 employee or either they have 2 digit emp-Id or 5 digit. Thus we will be using only 100 location in the array. Therefore, in order to keep the array size down, another good option is to use just last two digit of the key to identify each employee. for example 79439 → use 39 as Index of Array.

In this solution, we need a particular way to convert a five digit no. to a two digit number for array Indexing.

So we need a function, which will do the transformation.

this function is term as hash function and this array term as hash table.

Hash Table - is a data structure in which keys are mapped to array Position by a hash function. A value stored in hash table can be search or access in O(1) time using a hash function.

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

* The storage requirement for a hash table is just $O(K)$ where K is the number of keys actually used.

* In a hash table, an element with key K is stored at index $h(K)$ and not K. It means a hash function h is used to calculated the index. Thus the ~~process~~ process of mapping the key to appropriate locations in a hash table is called hashing. when two different keys generate the same key values of Array Index. This is known as collision.

Hash function

A Hash function is simply a mathematical formula which when applied to a key, produces an integer, which can be used as an index for the key in the hash table.

* Aim - element should be relatively, randomly and uniformly distributed.

* Hash function choose that reduce the no. of collisions.

* Properties of Good hash function
  - Low Cost
  - Determinism
  - Uniformity

Different Hash function-

1) Division Method

$h(z) = z \bmod M$

* Simplest method
* divide z by m and then uses the remainder thus obtained.

$h(z) = 4562 \bmod 100 = \boxed{62}$

Ex calculate hash value of keys 1234, 5462, setting m = 97

$h(1234) = 1234 \% 97 = 70$

$= 16$

(2) Multiplication Method

$h(x) = \lfloor m (kA \bmod 1) \rfloor$

A is a constant b/w $0 < A < 1$

m is the total no. of indices

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

As per knuth, the best choice of A is (sqrt 5 − 1)/2 = .6180339887

Ex hash table size 1000
map the key 12345

A = .618033, m = 1000 & k = 1234

= ⌊1000 (12345 × 0.618033 mod 1)⌋

= ⌊1000 (7629.617385 mod 1)⌋

= ⌊1000 (0.617385)⌋

= 617.385

= 617

(3) Mid square method

Ex k = 1234, k^2 = 1522756 = 27

k = 5642 k^2 = 31832164 = 21

① Square the value of key
② Extract the middle r bits of the result.

for Ex 100 memory location then r = 2

Folding Method -

step (1) Divide the key value into a no of parts.

(2) Add individual parts

Ex If the hash table has a size of 10000 then we need at least three degit in each part.

Eg Given a hash table of 100 locations, find hash value for 5678, 321 & 34567

Loc = 100 → so k_1, k_2 ... are 2 digit

5678 – 56 + 78 – 134 – (1)34 } Hash value
321 – 32 + 1 – 33 – Ignore
33
34567 – 34 + 56 + 7 → 97 – 97

Collision

Collision occur when the hash function map two different keys to same location Therefore, a method used to solve the problem of collision, also called collision resolution technique.

Two method – (1) Open addressing
(2) Chaining

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

(1) Collision Resolution by open Addressing

* to computes new position, after collision
* It use probe sequence
* All values are stored in the hash table.
* Contain two types of values -
  - sentinel values (-1) - to indicate empty location
  - data values - that show already there is a value, then other slots are examined systematically in the forward direction to find a free slot
  - If not a single position is found → overflow condition

* The process of examining memory location in hash table is called probing.

* Open Addressing technique
  - ↳ linear probing
  - ↳ quadratic probing
  - ↳ double hashing

Linear Probing

* simplest approach
* new hash value -

$$h(k, i) = (h'(k) + i) \bmod m$$

$k \bmod m$ — probe no. — size

$h'k+1$ - occupied
$h'k+2$ — "
$+3$ — "
$+4$ — "

Until a free location is found.

Ex Consider a hash table of size 10. Using linear probing, insert the keys 72, 27, 36, 24, 63, 81, 92 and 101 into the table.
Let $h'(k) = k \bmod m$, $m = 10$

key - 72

72 mod 10 = 2
27 mod 10
36 mod 10
24 mod 10
63
81
92
101 mod 10 = 1,

0	
1	81
2	72
3	63
4	24
5	92
6	36
7	27
8	101
9	

Quadratic Probing

following hash function is used to resolve the collision -

$h(k, i) = [h(k) + c_1 i + c_2 i^2] \bmod m$

i — is probing no from 0 to m-1

Eg Hash table of size 10.

Insert 72, 27, 36, 24, 63, 81, and 101.

$c_1 = 1, \quad c_2 = 3$

Key = 72

$[72 \bmod 10 + 1 \times 0 + 3 \times 0] \bmod 10$

$(72 \bmod 10) \bmod 10$

$= 2 \bmod 10$

$= 2$

key - 27 → 7

key → 36 → 6

key - 24 → 4

key → 63 → 3

key → 81 → 1

key → 101 → 1 - collision occur

$[101 \bmod 10 + 1 \times 1 + 3 \times 1] \bmod 10$

$[1 + 1 + 3] \bmod 10 \Rightarrow 5 \bmod 10$

$\Rightarrow 5$

Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel.: +91-0141-5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

असतो मा सद्गमय

Double Hashing

$$h(k, i) = [h_1(k) + i\, h_2(k)] \bmod m$$

h_1, h_2 — two hash function, $h_1(k) = k \bmod m$

m — Size of table $h_2(k) = k \bmod m'$

$m' < m$

Ex Consider a hash table of size = 11. Using double hashing, insert the keys 72, 27, 36, 24, 63, 81, 92 and 101

Let $m = 11$.

step 1: key 72

$$h(72,0) = [72 \bmod 10 + 0 \times 72 \bmod 8] \bmod 11$$
$$= [2 + (0 \times 0)] \bmod 10$$
$$= 2 \bmod 11$$
$$= 2$$

(2) 27 → 7

36 → 6

24 → 4

63 → 3

81 → 1

92 → 2 — collision occur

new

$h(92, 1) =$

$[92 \bmod 10 + (1 \times 92 \bmod 8)] \bmod 10$

$2 + (1 \times 4) \bmod 10$

$2 + 4 \bmod 10$

$6 \bmod 10$

6

$h(92, 2) = 92 \bmod 10 + (2 \times 92 \bmod 8] \bmod 10$

$2 + (2 \times 4) \bmod 10$

$2 + 8 \bmod 10$

$10 \bmod 10$

0 —

$h(101, 0) \rightarrow 1$

$(101, 1)$ — 6

2 — 1

3 6

4 — 1

Although double hashing is very efficient algorithm it always require m to be a prime number. In our case m = 10, hence the degradation in performance.

(2) Collision Resolution by Chaining

In chaining, each location in the hash table stores a pointer to a linked list that contains all the key values that were hashed to same location.

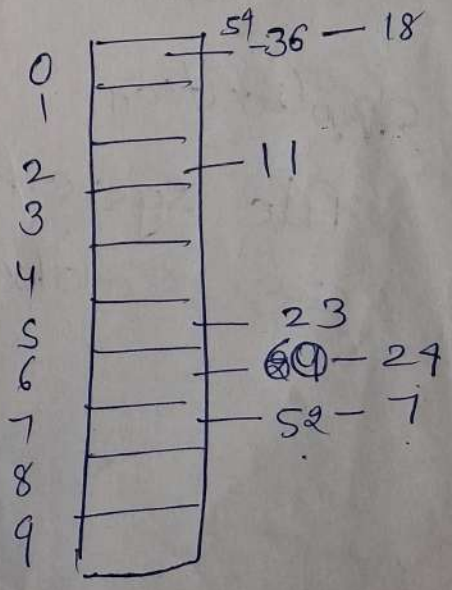
Swami Keshvanand Institute of Technology, Management & Gramothan,
Ramnagaria, Jagatpura, Jaipur-302017, INDIA
Approved by AICTE, Ministry of HRD, Government of India
Recognized by UGC under Section 2(f) of the UGC Act, 1956
Tel. : +91-0141- 5160400 Fax: +91-0141-2759555
E-mail: info@skit.ac.in Web: www.skit.ac.in

Ex Insert the keys 7, 24, 18, 52, 36, 54, 11, 23 and 60 in chained hash table of 9 memory locations. Use $h(k) = k \bmod m$.

Case $m = 9$

key(7) = 7 mod 9 = 7
key(24) = 24 mod 9 = 6
18 mod 9 = 0
52 mod 9 = 7
36 mod 9 = 0

54 mod 9 = 0
11 mod 9 = 2
23 mod 9 = 5
60 mod 9 = 6



Pros and Cons

* Remains effective even when no. of key values stored is much higher
* But this increment decrease the no. of performance.
* This technique is absolutely free fersem clustering.
* It inherit the disadvantage of list.
  (Extra space for next pointer)
  traversal- sequence

Application

file system

DB database